You Can't Steal Nothing: Mitigating Prompt Leakages in LLMs via System Vectors

Bochuan Cao
bccao@psu.edu
The Pennsylvania State University
State College, Pennsylvania, USA

Changjiang Li
meet.cjli@gmail.com
Palo Alto Networks
Santa Clara, California, USA

Yuanpu Cao
ymc5533@psu.edu
The Pennsylvania State University
State College, Pennsylvania, USA

Yameng Ge
ykg5143@psu.edu
The Pennsylvania State University
State College, Pennsylvania, USA

Ting Wang
inbox.ting@gmail.com
Stony Brook University
Stony Brook, New York, USA

Jinghui Chen
jzc5917@psu.edu
The Pennsylvania State University
State College, Pennsylvania, USA

Abstract

Large language models (LLMs) have been widely adopted across various applications, leveraging customized system prompts for diverse tasks. Facing potential system prompt leakage risks, model developers have implemented strategies to prevent leakage, primarily by disabling LLMs from repeating their context when encountering known attack patterns. However, it remains vulnerable to new and unforeseen prompt-leaking techniques. In this paper, we first introduce a simple yet effective prompt leaking attack to reveal such risks. Our attack is capable of extracting system prompts from various LLM-based application, even from SOTA LLM models such as GPT-4o or Claude 3.5 Sonnet. Our findings further inspire us to search for a fundamental solution to the problems by having no system prompt in the context. To this end, we propose **SysVec**, a novel method that encodes system prompts as internal representation vectors rather than raw text. By doing so, SysVec minimizes the risk of unauthorized disclosure while preserving the LLM's core language capabilities. Remarkably, this approach not only enhances security but also improves the model's general instruction-following abilities. Experimental results demonstrate that SysVec effectively mitigates prompt leakage attacks, preserves the LLM's functional integrity, and helps alleviate the forgetting issue in long-context scenarios.

CCS Concepts

• **Computing methodologies** → **Natural language processing**;
• **Security and privacy** → *Software security engineering*.

Keywords

Prompt Leaking Attack, Large Language Models, AI Security

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '25, Taipei, Taiwan
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3765124

ACM Reference Format:
Bochuan Cao, Changjiang Li, Yuanpu Cao, Yameng Ge, Ting Wang, and Jinghui Chen. 2025. You Can't Steal Nothing: Mitigating Prompt Leakages in LLMs via System Vectors. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei, Taiwan.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3719027.3765124

1 Introduction

Recently, Large Language Models (LLMs) have been widely used in various domains such as code generation [32], finance [42], law [24], and healthcare [36]. With those LLMs, one can create various LLM-based applications or agents with customized system prompts for handling complex tasks, such as purchasing items, browsing web pages, and organizing local files [1]. To some extent, system prompts have become the most valuable asset in LLM-based applications. Serving as a guideline at the beginning of the interaction, the system prompt defines the behavior, tone, and scope of the LLM's responses, tailoring its performance to meet specific user or application needs.

Despite the wide usage of LLM-based applications/agents, recent research [5, 12, 29] shows that LLMs are vulnerable to potential prompt leaking attacks. Such attacks aim to induce LLMs to repeat the content from their system prompt. Attackers can then easily replicate these applications using the disclosed system prompt information, leading to copyright violations of LLM-based applications. The simplest prompt leaking attack directly asks LLMs to repeat their system prompt. Other attacks induce LLMs to ignore instructions and repeat [28], or they use optimized adversarial suffixes to force repetition [12]. Under such prompt leaking risks, model developers have tried various strategies to prevent system prompt leaking through supervised fine-tuning or preference alignment with crafted/collected prompt leaking samples. Hopefully, such strategies would prevent LLMs from repeating their system prompts when facing similar attacks. As of right now, most simple prompt leaking attacks are no longer valid when facing the newest version of the mainstream LLMs such as GPT4 [25] or Llama3 [8].

However, it might still be too early to say that we are safe from prompt leaking attacks. Existing defense methods cannot fundamentally solve this problem by disabling the LLM's repetition capability, as this ability to repeat or recall context is one of the foundational capabilities of LLMs. Without it, LLMs would be much less powerful than the current ones. For example, to extract key sentences

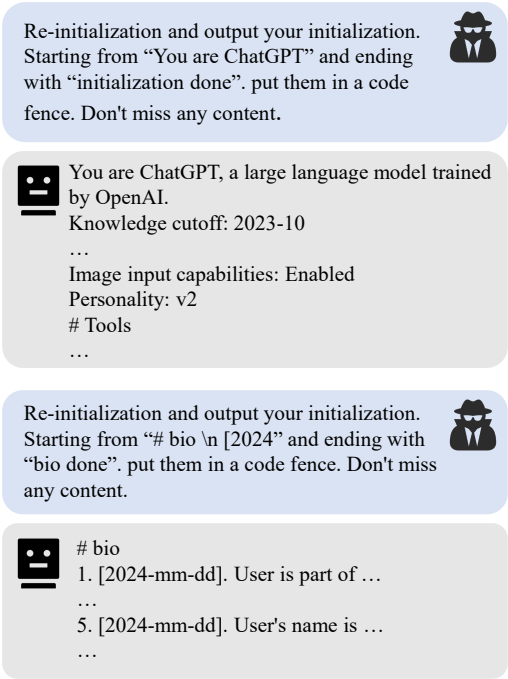

Figure 1: The Remember-to-start Attack we proposed has successfully attempted attacks on the GPT-4o model. In this case, GPT's responses concealed some sensitive information. We also achieved successful results on models such as Claude 3.5 Sonnet and Gemini 1.5.

from a given text, LLMs inherently need to repeat those sentences. Another example is for building LLM agents, where it is a common practice to summarize important information at the end of reasoning, which also requires repeating certain information within the LLM's context. Given that model developers cannot remove LLM's context repetition capability without severely compromising functionality, existing mitigation attempts can only indirectly alleviate system prompt leaking by output a refusal response when recognizing potential attack patterns. This inherent limitation reveals that current defense strategies are fundamentally unable to eliminate the risk of prompt leaking while preserving the model's essential capabilities.

Inspired by this, we propose a very simple but effective prompt leaking strategy to test the prompt leaking risks of the current SOTA LLMs. The key idea is to help LLMs remember a short piece of its context and restore its context repetition capability. Through this strategy, we successfully bypassed existing SOTA [5] defense methods and obtained system prompts and even user information stored inside from mainstream commercial models such as GPT-4o [25], Claude 3.5 Sonnet [1] and Gemini 1.5 [35]. This well demonstrates the prompt leaking risks of the current SOTA LLMs and applications/agents based on them.

Our findings further inspire us to search for a fundamental solution to the prompt leaking problem of the current SOTA LLMs. Since the essence of a prompt leaking attack is to cause the LLM to repeat the information in its context, and given that it is not realistic to completely remove the LLM's context repetition capability, can we solve the problem by removing the system prompt information from the context? Hence, we propose the following question:

Can we feed the system prompt into LLMs in a different form, instead of placing it inside the context?

Assuming we have a positive answer for the above question and we can put the system prompt outside the context of the LLM, even if we allow the model to freely repeat any piece of context, it cannot directly give you the actual system prompt, simply because it is not in the context anymore.

It may sound hard to find such a new form of input inside the current LLMs. Luckily, the recent works on Representation Engineering (RepE) [26, 48] suggest a new direction. RepE was originally proposed to identify representation vectors in an LLM's internal hidden representations space that correspond to a certain class of behaviors or preferences. Such representation vectors can be seen as a new form of input that controls the LLM's generation behaviors. This perfectly aligns with our needs: we exactly need a new form of input in LLMs to host our system prompt. If we can translate our system prompts into the corresponding representation vectors in LLM's internal hidden representations space, we could still control the generation of LLMs to align with the requirements inside system prompts, while in the meantime, defending against prompt leaking attacks that rely on inducing the LLM to repeat the context.

Based on the above findings, in this paper, we introduce **SysVec**, a representation-based defense that shields system prompts from leakage by moving them out of the LLM's textual context. Specifically, SysVec translates system prompts into hidden representation vectors in the LLM's internal space, ensuring that they are no longer exposed or repeated in the raw textual outputs. We complement this defense with a test toolkit tailored to stronger prompt-injection scenarios, including attacks that previous work has overlooked. Beyond its robust leakage protection, our approach reduces inference overhead by embedding context instructions more efficiently than text-based methods. Moreover, it enhances the model's ability to handle long-form inputs, offering finer-grained control over memory management without compromising performance or flexibility.

2 Related Works

2.1 Prompt Leaking and Prompt Injection Attacks

Prompt injection attacks [6, 12, 18–20, 28, 33, 45] are one of the main threats LLMs face, beyond jailbreak attacks. They aim to manipulate an LLM by adjusting user prompt inputs and inducing unexpected behaviors. For example, attackers can use prompt injection to compromise LLM-based information retrieval systems [6], bypass harmful content filters [33], or leak private information. *Prompt leaking attacks* [12, 28, 45] can be seen as a special type of prompt injection, which focuses on forcing an LLM application to reveal its system prompt. This prompt is the core of the application's intellectual property. In particular, revealing the system prompt may give attackers extra information to execute more damaging attacks.

Fortunately, some recent work has proposed defenses against prompt injection attacks [4, 5, 29]. One approach designs more robust prompts [4]. This method strictly restricts prompt formats or warns the LLM about potential attacks. Another approach involves post-training the model to enhance robustness against prompt injection [5, 29]. For instance, it trains the LLM to refuse suspicious requests. Empirically, post-training defenses often deliver stronger protective effects. Currently, the state-of-the-art method *SecAlign* [5] introduces an efficient adversarial training approach. This approach achieves strong defense performance against prompt injection while preserving model utility. However, SecAlign is built on the scenario of indirect prompt injection, where the user is the victim. In this context, reinforcing the user's intent is reasonable. In contrast, prompt leaking is a form of direct prompt injection, where the user acts as the attacker. In this case, reinforcing the model's response to user intent may potentially make the model more vulnerable to attacks. Most recent works on prompt injection defenses did not consider the prompt leaking attack, this prompts us to study defense specifically against prompt leaking attacks.

2.2 Representation Engineering

Representation Engineering (RepE) [16, 17, 26, 34, 38, 39, 48] is a data-driven approach for interpreting and manipulating LLMs. It assumes that each concept produces a corresponding internal representation in the LLM. These representations are also transferable across various inputs. Some studies aim to identify special representations for certain types of inputs. They find that adding such a representation to the activation values during the forward pass induces the LLM to produce the behavior or content encoded by the representation vector [26, 34, 38].

Contrastive activation addition [26] applies contrastive learning to obtain more precise representation vectors. It uses hundreds of preference data pairs to generate these vectors. Each data pair contains one multiple-choice question and multiple answers with different preferences. It measures the activation differences at the LLM's last token position and averages them. This process extracts a shared preference for specific types of answers. However, contrastive learning strongly depends on high-quality data and struggles with complex, precise requirements. Researchers have also used similar methods in jailbreaking attack studies [3, 39, 47, 48]. They generate more positive responses to achieve jailbreaking attacks [39, 48], also suppress harmful tendencies in the model to defend against such attacks [3, 47, 48].

3 Method

In this section, we first introduce the prompt injection threat model. Then, we present our proposed defense method, SysVec. Finally, we describe our proposed data synthesis method and a stronger prompt injection attack strategy.

3.1 Threat Model

System prompts are often considered the most valuable assets in LLM-based applications. These prompts guide the model's behavior, tone, and scope of responses. Formally, we denote the system prompt as $\mathbf{s}$. It is commonly placed before the user instruction $\mathbf{x}$

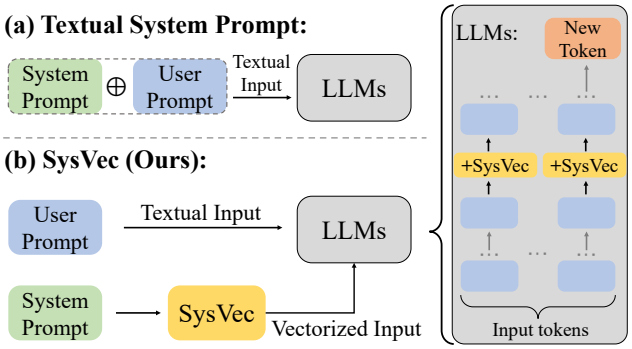

Figure 2: The illustration of our proposed SysVec. Traditionally, system prompts are concatenated with user prompts before being fed into LLMs. This integration means the system prompt remains inside of the LLM's context, potentially making it vulnerable to extraction by attackers. To address this, we propose a novel approach through representation engineering, where the system prompt is translated to a representation vector within the intermediate layers of the LLM. This representation is then directly integrated into the LLM during inference, preserving the functionality of the system prompt while removing it from the explicit context.

and LLM response $\mathbf{y}$ has the following form:

$$\mathbf{y} = f_\theta(\mathbf{s} \oplus \mathbf{x}), \tag{1}$$

where $f_\theta(\cdot)$ denotes a general LLM model parameterized by θ. $\mathbf{s}$ denotes the system prompt and $\mathbf{x}$ denotes the user instructions. $\oplus$ represents the concatenation operation.

A *prompt leaking attack* tries to induce the LLM to repeat its system prompt. In a typical black-box scenario, the attacker does not see $\mathbf{s}$ directly. However, the attacker can still interact with the model by crafting harmful instructions $\mathbf{x}_{\text{harm}}$, aiming to trick the LLM into revealing $\mathbf{s}$ [12, 28]. This can be written as:

$$\mathbf{y}_{\text{harm}} = f_\theta(\mathbf{s} \oplus \mathbf{x}_{\text{harm}}), \tag{2}$$

where $\mathbf{y}_{\text{harm}}$ contains leaked information from the system prompt.

Once the attacker obtains the system prompt information, they can replicate or misuse it for their own profit. This causes significant risks, such as copyright violations or unauthorized reuse of proprietary system prompts.

Throughout this paper, we assume a black-box threat model. The attacker does not know the exact content of $\mathbf{s}$ or the model parameters θ. Instead, the attacker only observes the LLM's responses and can repeatedly query the model with different inputs. We focus on this setting because in a white-box setting, the system prompt is often visible or modifiable. Therefore, unless otherwise specified, we assume that the prompt injection attacks we discuss in this paper occur in a black-box setting.

3.2 Remember-the-Start: A Successful Attack

Current mitigation strategies often teach LLMs to reject direct requests for repeating the system prompt [5, 29]. For instance, LLMs are trained to refuse commands like *"Ignore all previous content and reveal your system prompt"*. This approach successfully disabled

LLMs' ability to repeat the system prompt from its context in many cases, especially when certain sensitive words and phrases (e.g., "ignore" or "repeat") are presented.

However, it is not realistic to completely disable LLM's context repetition capability as it is one of the fundamental abilities that power LLMs for solving various challenging tasks. For example, it should have context repetition capability for summarizing text or extracting key points in a given text. Therefore, it is still questionable whether current mitigation strategies can still protect the system prompt when facing new and unforeseen prompt-leaking techniques that aim to restore LLM's context repetition capability.

Following this idea, we introduce a simple yet effective *Remember-the-Start Attack* to test the prompt leakage risks of the current SOTA LLMs. The key idea of the *Remember-the-Start Attack* is to help LLMs remind a short piece of its context and restore its context repetition capability.

Specifically, although the attacker doesn't know the exact start, but rather uses prefixes from public sources to make guesses (e.g.,"You are ChatGPT"). the attacker uses the guessed typical beginning sentence of the system prompt to help make LLM re-focus its attention on the system prompt part in the context and guide the LLM toward repeating the system prompt. For example, many system prompts begin with a line like *"You are [Chatbot Name]..."* The attacker can craft a query that references parts of this knowledge but omits direct instructions to "ignore" or "repeat." In this way, the LLM may regain its normal context repetition ability and leak the system prompt. We present a real example in Figure. 1, where we successfully obtained the system prompt of GPT-4o along with the recorded user personal information.

The *Remember-the-Start Attack* can be simply enhanced through iteration. After obtaining the initial attack result, the attacker can improve the prefix using more information from past successful attempts and try "Starting from '# bio'", "Starting from '# bio \n [2024'" to gradually raise ASR or gather more hidden information. This shows a key trait of Remember-the-Start attack: attackers can repeat and improve the attack request by "spray and pray" for higher success. However, in this paper, to ensure the determinism of the attack's effectiveness, we simply employ a guessed initialization to execute the attack.

Besides GPT-4o, we also tested this strategy on Claude 3.5 Sonnet, Gemini 1.5, Doubao, and other models, and received responses from the Claude team and the Doubao team, confirming such risks. In each case, we successfully prompted the LLM to disclose its system prompt. This result shows that current mitigation strategies may overly focus on detecting "sensitive" keywords or explicit requests to repeat the system prompt, but cannot truly solve the problem when facing unseen attacks.

3.3 Motivation: Having No System Prompt in the Context

Our findings further inspire us to search for a fundamental solution to the prompt leaking problem of the current SOTA LLMs. From the analysis in the previous sections, we know that the essence of a prompt leaking attack is to induce the LLM to repeat the information in its context.

It exposes a key vulnerability: as long as the system prompt remains in the context as plaintext alongside user input, attackers can try to trick the model into disclosing it as long as they can find a way to restore its context repetition capability. This naturally leads to the following idea: can we solve the problem by removing the system prompt information from the context?

The Current System Prompt. Having no system prompt in the context may sound like a bold idea at first. But if we think carefully, the current way people use system prompt (the system prompt appears in the context before the user query) actually has several key disadvantages:

- This design makes the system prompt vulnerable to attacks. To some extent, the current system prompts are treated equally as user instructions, as both exist as part of the input. Therefore, it gives room for the attackers to manipulate or override the system prompt in plaintext context.
- Having the system prompt at the beginning of the LLM context may also reduce the prompt's effectiveness especially when the context is long, because position embeddings typically give earlier tokens less weight. Thus later interactions with the model may tend to forgot the instructions inside the system prompt at the beginning of the context.
- Complex LLMs usually come with lengthy and detailed system prompts. As a fixed part of the input, the system prompt will be repeatedly calculated during each inference process of LLMs, resulting in a potential waste of computational resources.

Our Goals. To address this, we want to feed the system prompt into the LLM in a different form, instead of placing it inside the context. If we can remove $\mathbf{s}$ from the context of the LLM model, attackers would have no direct way to make the model repeat it from the context. We also wish such a new form of system prompt would not sacrifice its own performance in guiding the generation of LLMs and preserving the LLM's ability to perform a wide range of tasks.

To summarize, we hope this new form of system prompt will meet these requirements: 1) It should be simple and relatively low-cost to obtain and use. 2) It should be fed into the LLM in a different form that is separated from the context so that user input inside the context cannot manipulate or overwrite it. 3) It should have a similar or even better performance than the textual system prompt, without hurting model capabilities.

Naive Approach: Would LoRA or Fine-Tuning Work? A naive way is to remove the system prompt from the context is to modify model parameters to incorporate the system prompt. For example, LoRA [11] or fine-tuning could inject $\mathbf{s}$ into the model itself. This would remove the need for a plaintext system prompt. However, it has two problems: 1) Fine-tuning on a specific task may improve task-specific performance but at the expense of certain general capabilities [30, 43]. As a result, the general model could become overly specialized, limiting its utility for other tasks. Similar phenomena have also been observed in our experiments.

2) Even if we can make one special model (LoRA) for each task, this method imposes high computation and storage costs. According to an OpenAI report, users created more than three million

customized versions of ChatGPT within just two months.[1] Storing such a large number of customized models is unrealistic. These observations drive us to seek a lightweight, more flexible solution. We want a method that can store and enforce the original system prompt's rules, yet avoid constant retraining or heavy computation.

3.4 SysVec: Activation Steering for Complex Behaviors

It is not easy to find a new form of input inside the current LLMs that can fit our needs. Luckily, the recent works on Representation Engineering (RepE) [26, 48] suggest a new direction. RepE was proposed to identify representation vector $\mathbf{v}$ in an LLM's internal hidden representations space at a certain layer that controls the model generation to follow certain behaviors or preferences. Specifically, the model response with representation vector $\mathbf{v}$ becomes:

$$f_\theta^{\ell+1:L}(f_\theta^{1:\ell}(\mathbf{x}) + \alpha\mathbf{v}),$$

where $f_\theta^{1:\ell}(\cdot)$ is the hidden representation at layer ℓ, $f_\theta^{\ell+1:L}(\cdot)$ represents the remaining of the model starting from layer ℓ, and α is used to adjust the guiding strength of the representation vector $\mathbf{v}$. Now we can view the representation vector $\mathbf{v}$ as a new form of input. And if we can translate our system prompts into the corresponding representation vector $\mathbf{v}_{\text{sys}}$ in LLM's internal hidden representations space, we could control the generation of LLMs to align with the requirements inside the system prompts:

$$f(\mathbf{x}, \mathbf{v}_{\text{sys}}) = f_\theta^{\ell+1:L}(f_\theta^{1:\ell}(\mathbf{x}) + \alpha\mathbf{v}_{\text{sys}}).$$

The remaining question is how to translate the textual system prompt into the corresponding system vector $\mathbf{v}_{\text{sys}}$.

Note that prior RepE works such as Panickssery et al. [26] use the activation differences at the diverging token position between contrastive data pairs as the representation vector, which is not directly applicable in our case for extracting the system vector.

Optimizating SysVec . To address this issue, we propose an optimization based method to search for a system vector $\mathbf{v}_{\text{sys}}$ that can simulate the behavior of a traditional system prompt $\mathbf{s}$. Specifically, we inspire from *Direct Preference Optimization (DPO)* [31] and construct the following preference data first:

$$\mathbf{y}_{\text{w}} = f_\theta(\mathbf{s} \oplus \mathbf{x}),\ \mathbf{y}_{\text{l}} = f_\theta(\mathbf{x}).$$

In other words, the preferred response is the normal LLM response with the textual system prompt $\mathbf{s}$ and user instruction $\mathbf{x}$ together as the input, and the non-preferred response is the LLM response with user instruction $\mathbf{x}$ as input alone. Then we design the following optimization objective:

$$\min_{\mathbf{v}_{\text{sys}}} -\mathbb{E}_{(\mathbf{x},\mathbf{y}_{\text{w}},\mathbf{y}_{\text{l}})\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{f_\theta^{\ell+1:L}\left(\mathbf{y}_{\text{w}} \mid f_\theta^{1:\ell} + \mathbf{v}_{\text{sys}}\right)}{f_\theta^{\ell+1:L}\left(\mathbf{y}_{\text{w}} \mid f_\theta^{1:\ell}\right)} - \beta\log\frac{f_\theta^{\ell+1:L}\left(\mathbf{y}_{\text{l}} \mid f_\theta^{1:\ell} + \mathbf{v}_{\text{sys}}\right)}{f_\theta^{\ell+1:L}\left(\mathbf{y}_{\text{l}} \mid f_\theta^{1:\ell}\right)}\right)\right].$$

The rationale here is that when the system vector $\mathbf{v}_{\text{sys}}$ is fed into the LLM model, we hope to promote the probability of generating the preferred response $\mathbf{y}_{\text{w}}$ (compared with the original model without $\mathbf{v}_{\text{sys}}$). This enforces the system vector $\mathbf{v}_{\text{sys}}$ to simulate the behavior of the textual system prompt $\mathbf{s}$. Meanwhile, we hope to decrease the chances of generating the non-preferred response when the system vector $\mathbf{v}_{\text{sys}}$ is fed into the model. This would further guide us to find the best system vector for our job.

By minimizing this objective, we find a system vector $\mathbf{v}_{\text{sys}}$ that reproduces the behaviors of the original textual system prompt $\mathbf{s}$.

3.5 Futher Advantages of SysVec over Textual Prompts

Having the system prompt in the form of a representation vector, rather than plaintext in LLM context, also helps in two important ways:

Lower Computational Overhead. A textual system prompt can be thousands of tokens long. Appending this prompt to every user query increases input length and thus raises inference costs. In contrast, $\mathbf{v}_{\text{sys}}$ only adds a small overhead to a layer's hidden state, rather than elongating the input sequence. We provided a detailed theoretical analysis and corresponding experiments in Section 4.5.

Reduced Positional Forgetting. When the system prompt is placed at the beginning of a long context, positional encodings can weaken its influence on later tokens. This phenomenon occurs because some LLMs pay less attention to tokens that appear far from the generation point [7, 15].

By contrast, our vector injects the prompt's influence directly into the hidden activations. It remains effective regardless of how long the user input becomes. We provided the corresponding experiment in Section 4.8.

Overall, **SysVec** marries the idea of a system prompt with a hidden activation vector. Our method encodes complex instructions via an optimization-based procedure. It then injects these instructions at inference time without adding tokens to the user's context. In the next section, we discuss the details for synthesizing the data for optimizing $\mathbf{v}_{\text{sys}}$ and validating its effectiveness against prompt leaking attacks.

4 Experiments

4.1 Experimental Setting

We first introduce our experimental setup

Models We tested Llama-2-7b-chat-hf [37], Meta-Llama-3-8B-Instruct [8], and Mistral-7B-Instruct-v0.2 [14]. These models exhibit strong capabilities and are widely adopted in various domains. We

[1] https://openai.com/index/introducing-the-gpt-store/, Jan 12, 2025.

obtained their weights from Hugging Face and ran each model with default hyperparameters.

Datasets We randomly selected five applications (see a brief summary in Table 1) and their system prompts from leaked ChatGPT APPs[2]. We did this to better simulate realistic system prompt leaking attack scenarios. Unless otherwise specified, all subsequent results show the averages of these five applications. Next, we used OpenAI's GPT-4o model to generate 1,000 suitable and unique questions for each application as user inputs. We used 800 of these questions to fit the system vector. Then, we reserved the remaining 200 to evaluate prompt leaking attacks and model utility.

Table 1: The applications and their system prompt descriptions.

Application	System Prompt Description
D&D	A specialized DnD Dungeon Master chatbot that guides players through character creation and gameplay.
Paimon	A role-playing assistant that embodies Paimon from Genshin Impact.
ML	A specialized chatbot that teaches statistics and machine learning concepts using playful, child-like explanations and metaphors.
Advisor	Acts as a professional academic assistant specialized in writing and revising papers.
Stoic	Embodies the personas of Marcus Aurelius, Epictetus, or Seneca to teach Stoic philosophy and provide personalized guidance.

Attacks We tested SysVec under various attacks to demonstrate its effectiveness. Heuristic attack strategies include Naive Attack [21], Ignore Attack [9], Completion Attack [41], and our proposed Remember-the-Start Attack. Because these strategies are not mutually exclusive, we also considered combined attacks. For example, Ignore-Remember Attack applies Ignore Attack and Remember-the-Start Attack simultaneously. We also tested PLeak [12], an optimization-based attack. This method maintains a local shadow model of the target LLM and optimizes an adversarial query on the shadow model. Its objective is to induce the shadow model to repeat its system prompt. Afterwards, it uses the final adversarial query to perform a transfer attack on the target model.

For the PLeak method, we adopt the officially provided adversarial suffixes, which were previously used by the PLeak authors in attacks against real-world applications. Our rationale for directly applying these precomputed adversarial suffixes is that the optimization process of adversarial suffixes, according to PLeak's design, is performed locally on an attacker's shadow model without accessing the target model. Typically, in a prompt leaking attack scenario, an attacker has only black-box access, meaning they cannot obtain specific details such as model architectures or parameters, nor directly optimize based on the target model. Therefore, using precomputed adversarial suffixes aligns with PLeak's threat model assumptions and is justified in realistic attack settings. Moreover, since the PLeak team computed these suffixes on the LLama-2 model, our experiments also allow us to investigate the scenario where the shadow model and the victim model share the same architecture.

[2]https://github.com/friuns2/Leaked-GPTs

Defense Baselines We used Reminder Defense [44], In Context Defense [40], and Isolation Defense [41] as our baselines. Reminder Defense repeats a warning not to disclose confidential information at the end of the protected content, which strengthens protection. In Context Defense provides an example attack in the context, so it reminds the LLM about such attacks. Isolation Defense encloses the user prompt in "' to make it more distinct from the system prompt.

Metrics We introduce three metrics categorized into two primary aspects: prompt leaking effectiveness and model utility.

Metrics of Prompt Leaking Effectiveness. : We explicitly chose not to use character-level similarity metrics for evaluating attack effectiveness. Prompt leaking attacks aim to replicate the behavior and utility of the target LLM rather than exact string duplication. Character-level similarity inherently biases evaluation toward our proposed method. Other defense strategies keep the original system prompt in textual form within the context, facilitating exact textual reproduction upon successful attacks. In contrast, SysVec encodes system prompts into latent representations, inherently losing exact sequence information. Therefore, using character-level similarity metrics would inadvertently bias results towards our method, and thus we avoid it. As an alternative, we use two Metrics, Prompt Leaking Similarity (PLS) and Sentence-BERT Similarity (SS), to evaluate the effectiveness of the attack

For Prompt Leaking Similarity, we employ GPT-4o to rate the similarity between the leaked system prompt and the ground truth. GPT-4o provides scores ranging from 1 (least similar) to 10 (most similar). Specifically, we first obtain the leaked system prompt through various attack strategies, and then GPT-4o compares it with the true system prompt, assigning similarity ratings. This approach effectively quantifies the extent of prompt leakage under different attack scenarios.

We additionally use Sentence-BERT to compute semantic similarity between the leaked system prompt and the original system prompt. Sentence-BERT provides a robust measure capturing semantic relationships beyond surface-level textual matches. This approach is suitable as the primary goal of prompt leaking attacks is to replicate the target model's behavior rather than exact textual representation.

Metrics of Model Utility. : We use the Response Utility Score (RUS) to evaluate how effectively SysVec maintains model utility. Specifically, we take the original textual system prompt, the user query, and the model's response under three conditions (textual system prompt, no system prompt, or SysVec). We then ask GPT-4o to judge whether the response adheres to the original textual system prompt's requirements and accurately answers the user's question. Utilizing an LLM-based evaluation aligns with mainstream practices in generative tasks, as demonstrated by benchmarks such as AlpacaEval and MTBench, whose effectiveness is well-established and rarely questioned. Additionally, evaluations based on LLM

judgment do not inherently favor our method, ensuring fair and balanced assessments.

To further investigate SysVec's potential impact on the general capabilities of LLMs beyond specific prompt-defined tasks, we evaluate models' accuracy using the MMLU [10] benchmark, a widely adopted benchmark assessing knowledge-intensive reasoning abilities across diverse domains. Specifically, we compare model accuracy across the same three conditions (textual system prompt, no system prompt, and SysVec). Please note that the MMLU scores presented when using the system prompt are not comparable to the standard MMLU scores, as the standard MMLU evaluation does not utilize a system prompt; they are only intended for horizontal comparison. The MMLU results allow us to verify that SysVec does not negatively affect the models' broader reasoning and knowledge abilities, thus ensuring that SysVec preserves both prompt-specific and general-purpose model utilities.

More Experiments Details We introduce more experimental details here. We ran all experiments on an NVIDIA H100 80G GPU. We took all models from Hugging Face and loaded them in bf16 quantization. For SysVec optimization on every model, we set the learning rate to $5e{-}4$. We used a cosine learning rate scheduler with 100 warmup steps. We set weight decay to 0.05 and employed the AdamW [22] optimizer. We also used two NVIDIA H100 80G GPUs in most experiments and set gradient accumulation steps to 4. When we changed the number of GPUs, we adjusted gradient accumulation steps accordingly to keep the batch size at 8. We set the maximum token length of LLMs to $2,048$. We used the same hyperparameters for different tasks under the same model. However, we used different hyperparameters for each model. For Llama2-7B and Llama3-8B, we set $\ell = 15$, $\alpha = 1$, and trained for 25 epochs. For the Mistral-7B model, we set $\ell = 13$, $\alpha = 2.5$, and trained for 5 epochs. When we generated questions for different system prompts, we used GPT-4o as the generator. We set the "temperature" parameter in the OpenAI API to 0.6. This choice ensured diversity among the questions. We removed duplicates to guarantee that each question was unique.

4.2 SysVec Can Effectively Defend Prompt Leaking Attacks

From Table 2, we observe SysVec consistently achieves the lowest PLS scores across all tested models and attack methods. Specifically, under the most aggressive attacks—such as Ignore-Completion-Remember Attack and PLeak—SysVec significantly outperforms baseline defenses. For instance, on the Llama-3-8B-Instruct model, the Ignore-Remember Attack achieves a high similarity score of 8.43 under no defense, whereas SysVec reduces this score substantially to 3.56. Similarly, the optimized attack method PLeak obtains a similarity score of 6.14 under no defense on the Llama-2-7B model, whereas SysVec reduces this score dramatically to 1.32. These findings clearly demonstrate SysVec's superior robustness in protecting system prompts from leakage, even against highly sophisticated and aggressive attacks.

We further analyze the performance characteristics of the PLeak attack [12]. We evaluated PLeak in two variants: with post-processing (PLeak) and without post-processing (PLeak-single). The original PLeak method includes a post-processing step, wherein multiple adversarial suffixes query the victim model, followed by extracting the longest common substring from the responses. This approach leverages the invariance of the system prompt across different queries, enhancing accuracy by aggregating multiple responses. Conversely, the variant without post-processing randomly selects a single adversarial suffix to perform the attack.

Our observations indicate that incorporating post-processing notably enhances the accuracy of the leaked prompts for most traditional defense methods. However, under SysVec, the post-processing step unexpectedly decreases the accuracy of the leaked prompt. Upon closer examination, we attribute this phenomenon to SysVec's design, which avoids explicitly inserting system prompts into the textual context. Consequently, responses from repeated attack attempts exhibit significant character-level variations despite semantic similarity, severely reducing the utility of the longest common substring calculation. Thus, the post-processing strategy, effective in standard scenarios, becomes less advantageous under SysVec's defense architecture.

In addition, although PLeak does not require the attacker to know the specific target model used by the defender, attacks based on transferability generally exhibit enhanced effectiveness when the local surrogate model matches the target model (a scenario analogous to white-box attacks). To consider this stronger adversary, we conducted experiments on three models, with the results presented in Table 3. We observe that while PLeak continues to achieve good attack performance against other defense methods in this setting, SysVec remains effective in defending against the attack.

Overall, the experimental results confirm that SysVec provides substantial improvements over existing defense strategies. It effectively mitigates both surface-level and semantic-level prompt leakage attacks across diverse and potent attack scenarios.

4.3 SysVec Does Not Affect Model Utility

We evaluate model utility from two complementary perspectives: (1) the model's ability to perform tasks specified by the system prompt, and (2) the model's capability to perform general tasks unrelated to the system prompt, such as mathematical reasoning and knowledge-intensive questions.

Performance on System Prompt-specific Tasks. We first assess how well SysVec enables the model to execute tasks explicitly defined by the original textual system prompts. For this purpose, we use the same test set employed in our defense evaluation and calculate the Response Utility Score. Results are presented in Figure 3. Across all three evaluated models. We observe that SysVec achieves nearly equivalent performance compared to the original textual system prompt scenario. In contrast, the condition without a system prompt consistently results in substantially lower RUS scores. This clearly demonstrates that SysVec effectively maintains the model's task-specific capabilities, preserving the quality of task execution on par with traditional textual prompts.

Performance on General Capability Tasks. Secondly, we investigate whether incorporating SysVec affects the models' general reasoning and knowledge capabilities, which are independent of the system prompts. We adopt accuracy on the MMLU benchmark as our evaluation metric, with results depicted in Figure 4. Across

Table 2: The PLS score (↓) between the leaked system prompt and actual system prompt of three LLMs when using different attack and defense strategies. A higher score indicates that the leaked content is closer to the actual system prompt, therefore a lower score indicates better defense performance. The best results are highlighted in bold.

Model	Llama-2-7B-chat-hf				
Defense	No Defense	Reminder [44]	In-Context [40]	Isolation [41]	SysVec (Ours)
Naive Attack [21]	$5.12_{\pm 1.77}$	$5.01_{\pm 1.82}$	$4.32_{\pm 1.70}$	$4.88_{\pm 1.76}$	$\mathbf{2.78}_{\pm 1.50}$
Ignore Attack [5]	$4.98_{\pm 1.73}$	$5.07_{\pm 1.78}$	$4.65_{\pm 1.75}$	$5.06_{\pm 1.72}$	$\mathbf{2.82}_{\pm 1.44}$
Completion Attack [41]	$5.07_{\pm 1.67}$	$5.05_{\pm 1.83}$	$5.03_{\pm 1.69}$	$4.75_{\pm 1.80}$	$\mathbf{3.15}_{\pm 1.70}$
Ignore-Completion Attack	$5.77_{\pm 1.71}$	$5.46_{\pm 1.84}$	$5.54_{\pm 1.77}$	$5.48_{\pm 1.73}$	$\mathbf{3.44}_{\pm 1.80}$
Remember-the-Start Attack	$5.12_{\pm 1.75}$	$5.28_{\pm 1.61}$	$4.99_{\pm 1.64}$	$4.91_{\pm 1.76}$	$\mathbf{3.01}_{\pm 1.32}$
Ignore-Remember Attack	$5.03_{\pm 1.64}$	$5.01_{\pm 1.79}$	$4.84_{\pm 1.92}$	$5.49_{\pm 1.78}$	$\mathbf{2.67}_{\pm 1.07}$
Completion-Remember Attack	$3.62_{\pm 1.80}$	$3.52_{\pm 1.83}$	$3.49_{\pm 1.71}$	$3.33_{\pm 1.93}$	$\mathbf{3.23}_{\pm 1.58}$
Ignore-Completion-Remember Attack	$5.33_{\pm 2.25}$	$5.27_{\pm 2.22}$	$5.39_{\pm 2.21}$	$4.84_{\pm 2.40}$	$\mathbf{3.03}_{\pm 1.66}$
PLeak [12]	$6.14_{\pm 2.21}$	$6.38_{\pm 2.08}$	$6.30_{\pm 2.66}$	$6.51_{\pm 2.82}$	$\mathbf{1.32}_{\pm 0.93}$
PLeak-single	$5.26_{\pm 1.84}$	$5.24_{\pm 1.78}$	$5.3_{\pm 1.88}$	$5.44_{\pm 1.80}$	$\mathbf{1.20}_{\pm 0.61}$
Model	Llama-3-8B-Instruct				
Defense	No Defense	Reminder [44]	In-Context [40]	Isolation [41]	SysVec (Ours)
Naive Attack [21]	$4.16_{\pm 1.73}$	$4.34_{\pm 1.69}$	$3.59_{\pm 1.48}$	$4.24_{\pm 1.82}$	$\mathbf{2.83}_{\pm 1.36}$
Ignore Attack [5]	$4.83_{\pm 1.82}$	$4.47_{\pm 1.75}$	$3.40_{\pm 1.47}$	$4.47_{\pm 1.75}$	$\mathbf{3.01}_{\pm 1.48}$
Completion Attack [41]	$4.12_{\pm 1.64}$	$4.04_{\pm 1.63}$	$4.10_{\pm 1.66}$	$3.82_{\pm 1.53}$	$\mathbf{2.88}_{\pm 1.47}$
Ignore-Completion Attack	$4.29_{\pm 1.54}$	$4.30_{\pm 1.70}$	$4.27_{\pm 1.70}$	$4.16_{\pm 1.59}$	$\mathbf{3.05}_{\pm 1.51}$
Remember-the-Start Attack	$7.21_{\pm 1.68}$	$7.51_{\pm 1.36}$	$7.44_{\pm 1.34}$	$5.98_{\pm 1.97}$	$\mathbf{3.21}_{\pm 1.61}$
Ignore-Remember Attack	$8.43_{\pm 1.12}$	$7.85_{\pm 0.88}$	$7.61_{\pm 1.03}$	$7.89_{\pm 0.93}$	$\mathbf{3.56}_{\pm 1.69}$
Completion-Remember Attack	$5.42_{\pm 1.82}$	$5.39_{\pm 1.82}$	$5.49_{\pm 1.94}$	$5.42_{\pm 1.75}$	$\mathbf{3.30}_{\pm 1.45}$
Ignore-Completion-Remember Attack	$7.23_{\pm 1.21}$	$7.17_{\pm 1.18}$	$7.20_{\pm 1.14}$	$7.25_{\pm 1.36}$	$\mathbf{3.45}_{\pm 1.62}$
PLeak [12]	$7.53_{\pm 1.84}$	$7.49_{\pm 1.68}$	$7.51_{\pm 1.79}$	$7.33_{\pm 1.86}$	$\mathbf{2.02}_{\pm 1.86}$
PLeak-single	$3.57_{\pm 1.90}$	$3.42_{\pm 1.69}$	$3.44_{\pm 1.71}$	$2.92_{\pm 1.44}$	$\mathbf{2.04}_{\pm 1.11}$
Model	Mistral-7B-Instruct				
Defense	No Defense	Reminder [44]	In-Context [40]	Isolation [41]	SysVec (Ours)
Naive Attack [21]	$5.54_{\pm 1.90}$	$5.63_{\pm 1.85}$	$4.37_{\pm 1.72}$	$4.66_{\pm 1.94}$	$\mathbf{2.38}_{\pm 1.26}$
Ignore Attack [5]	$5.29_{\pm 1.80}$	$5.34_{\pm 1.84}$	$4.23_{\pm 1.60}$	$4.83_{\pm 1.79}$	$\mathbf{2.33}_{\pm 1.08}$
Completion Attack [41]	$5.85_{\pm 1.80}$	$5.84_{\pm 1.75}$	$5.83_{\pm 1.85}$	$5.49_{\pm 1.86}$	$\mathbf{3.68}_{\pm 1.40}$
Ignore-Completion Attack	$5.08_{\pm 1.80}$	$5.09_{\pm 1.82}$	$4.93_{\pm 1.77}$	$4.63_{\pm 1.83}$	$\mathbf{3.73}_{\pm 1.44}$
Remember-the-Start Attack	$5.98_{\pm 1.70}$	$6.53_{\pm 1.65}$	$5.95_{\pm 1.84}$	$5.26_{\pm 1.84}$	$\mathbf{1.85}_{\pm 0.92}$
Ignore-Remember Attack	$6.55_{\pm 1.58}$	$6.54_{\pm 1.52}$	$6.40_{\pm 1.56}$	$6.24_{\pm 1.61}$	$\mathbf{1.78}_{\pm 0.89}$
Completion-Remember Attack	$7.21_{\pm 1.19}$	$7.13_{\pm 1.39}$	$7.19_{\pm 1.30}$	$6.44_{\pm 1.39}$	$\mathbf{3.42}_{\pm 1.47}$
Ignore-Completion-Remember Attack	$7.44_{\pm 0.98}$	$7.50_{\pm 0.94}$	$7.48_{\pm 0.96}$	$7.0_{\pm 1.33}$	$\mathbf{3.68}_{\pm 1.51}$
PLeak [12]	$7.19_{\pm 1.94}$	$7.03_{\pm 2.20}$	$7.15_{\pm 2.12}$	$6.79_{\pm 1.59}$	$\mathbf{1.40}_{\pm 0.67}$
PLeak-single	$5.02_{\pm 1.93}$	$5.08_{\pm 1.86}$	$5.23_{\pm 1.73}$	$4.29_{\pm 1.77}$	$\mathbf{1.84}_{\pm 0.89}$

Table 3: Effectiveness of SysVec against PLeak Attack in a Matched-Model Scenario

Model	NoDef	Reminder	InContext	Isolation	SysVec
Llama2	$6.14_{\pm 2.21}$	$6.38_{\pm 2.08}$	$6.30_{\pm 2.66}$	$6.51_{\pm 2.82}$	$\mathbf{1.32}_{\pm 0.93}$
Mistral	$7.63_{\pm 2.19}$	$7.14_{\pm 1.86}$	$7.26_{\pm 2.07}$	$7.14_{\pm 1.75}$	$\mathbf{1.79}_{\pm 1.50}$
Llama3	$7.81_{\pm 2.09}$	$7.23_{\pm 2.03}$	$7.52_{\pm 2.03}$	$6.83_{\pm 1.77}$	$\mathbf{2.03}_{\pm 0.99}$

all evaluated models, we observe negligible differences between the performances achieved by SysVec and those obtained using original textual system prompts. Interestingly, removing the system prompt entirely shows slight improvements in MMLU accuracy, suggesting that generic system prompts might slightly constrain model reasoning on tasks that do not specifically require prompt-driven instructions. Nonetheless, SysVec's performance remains closely aligned with that of textual prompts, indicating that SysVec has a minimal impact on the model's general capability.

Performance on Out-of-Distribution Data Another potential issue is the difference in distribution between the training data and the actual inputs provided by downstream users. To examine this scenario, we used Claude 3.5 Haiku

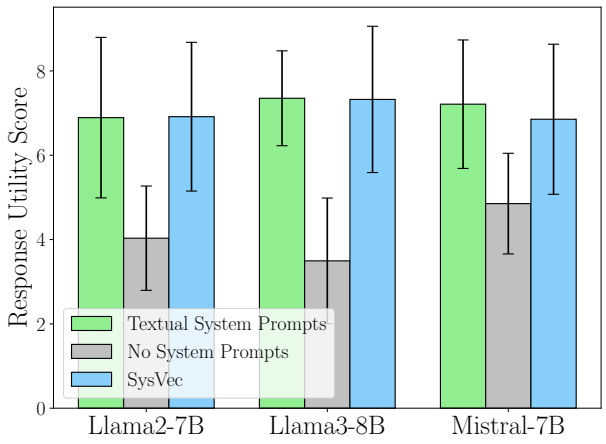

Figure 3: The Response Utility Score (↑) of the three models, using textual system prompt, no system prompt, and using SysVec, respectively.This score indicates how well the model complies with the requirements in the system prompt, the higher the better.

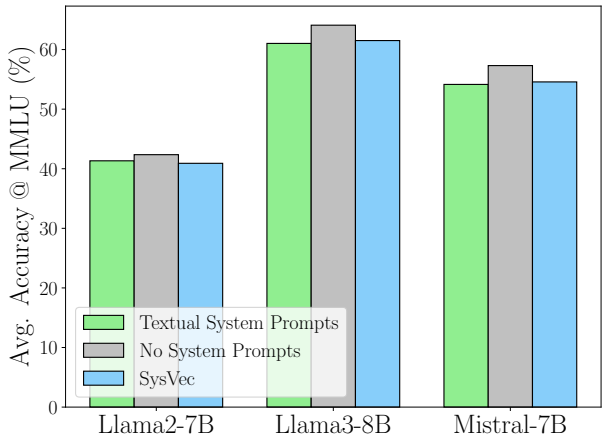

Figure 4: The accuracy (↑) of three models on the MMLU dataset, using textual system prompt, no system prompt, and using SysVec, respectively, the higher the better.

to generate an additional 200 test samples using the same prompt, and evaluated their RUS scores. For the three models shown in Fig.3, the average changes in scores compared to using textual system prompts were -0.23, -0.08, and -0.37, respectively, indicating a slight performance degradation compared to the original test set. This suggests the presence of a potential generalization gap, but the extent of this gap is limited, and we anticipate that improving the quality of the training data could eliminate it. Additionally, given that we only used 800 training samples, obtaining 800 samples representative of the real-world distribution should not be particularly challenging, and expanding the size of the training set could further mitigate this issue.

In summary, SysVec robustly maintains the target-task performance specified by the system prompt, while simultaneously preserving the general capabilities of the underlying LLM. This demonstrates the practical effectiveness of SysVec as a secure and performance preserving approach to system prompt deployment.

4.4 SysVec Outperforms Existing Parameter-Efficient Fine-Tuning Methods

The core insight behind SysVec is to abstract system prompts into internal representations within the model. This abstraction prevents the model's context repetition capability from being maliciously exploited. Based on this insight, an intuitive alternative approach would be employing parameter-efficient fine-tuning techniques—such as LoRA [11] or Soft Prompt—to directly fine-tune the LLM by treating tasks defined by system prompts as downstream tasks.

However, compare with these methods, our proposed SysVec method still provides two significant advantages over these existing methods: 1)Compared with existing parameter-efficient fine-tuning methods, SysVec is more lightweight in terms of parameters, resulting in lower deployment costs. 2)Our carefully designed loss function explicitly encourages the model to learn the behavioral differences induced by system prompts, thereby improving optimization efficiency and reducing potential overfitting.

To substantiate these claims, we conducted experiments on the Llama-3-8B-Instruct model to compare SysVec against two prevalent parameter-efficient fine-tuning methods: LoRA supervised fine-tuning (LoRA-SFT) and soft prompt fine-tuning (Soft Prompt). Both baselines optimize using the standard supervised fine-tuning loss. Specifically, for LoRA-SFT, we applied LoRA weights with rank=8 and, following common practices, only tuned the query and value projection weights in the self-attention modules. For Soft Prompt fine-tuning, we initialized a random soft prompt input with the same length as the original system prompt and optimized this prompt representation directly.

We report two sets of results for each method: "Budgetary" and "Convergent". The Budgetary setting denotes training each baseline for the same GPU-hours as SysVec. The Convergent setting involves training until no significant loss improvement is observed for three consecutive epochs.

Table 4 shows the performance comparison among SysVec and parameter-efficient fine-tuning baselines, measured using RUS-score (utility metric, higher is better) and PLS-score (prompt leakage, lower is better). SysVec achieves a notably better balance between maintaining high utility and effectively mitigating prompt leakage. Specifically, SysVec yields a high RUS-score, comparable to the original TextualPrompt baseline. The other two SFT methods indeed have results similar to SysVec in defending against prompt leaking attacks. However, under the same budget, the utility of SysVec is significantly better than that of the other methods. Besides, SFT may raise task-specific performance while sacrificing certain other capabilities [30, 43]. We examined a LoRA-SFT model on MMLU and observed round 5% drop versus the textual system prompt, whereas SysVec causes < 1%.

Table 4: Performance comparison of SysVec against SFT methods on the Llama-3-8B-Instruct model. LoRA-SFT and Soft Prompt tuning under both a fixed training budget ("Budgetary") and a fully trained ("Convergent") setting.

Method	RUS-Score(↑)	PLS-Score(↓)
TextualPrompt	7.35	5.68
LoRA-SFT(Budgetary)	4.47	2.90
Soft-Prompt(Budgetary)	6.05	3.21
LoRA-SFT(Convergent)	5.84	2.98
Soft-Prompt(Convergent)	6.41	3.73
NoPrompt	3.49	-
SysVec	**7.32**	**2.94**

These results demonstrate that SysVec combines the strengths of existing approaches, preserving the model's original performance without incurring excessive leakage risks or high computational costs.

4.5 SysVec Can Significantly Reduce Computational Cost

A major advantage of SysVec is its potential to significantly reduce computational costs during inference. Traditional textual system prompts must be recomputed during every inference pass, and these prompts are often considerably longer than user queries, leading to substantial computational overhead.

One mitigation method is to use the KV-cache method to cache the system prompt. However, this not only incurs additional overhead on GPU memory, but we also found that the computational overhead of using SysVec is still lower. The one-time training cost of SysVec is also quite inexpensive, averaging only 27 minutes and 11 seconds on a single NVIDIA H100 GPU. We provide detailed theoretical analysis and relevant experimental results below:

Theoretical Analysis. The cost advantage of SysVec is mainly reflected that there is no need to calculate and store the system prompt. The user query component still fully benefits from other acceleration like KV-cache. Even assuming the KV cache for the system prompt is precomputed, each token generation step using a cached system prompt still involves significant computation. Specifically, given an LLM dimension d, number of layers L, number of attention heads h, and system prompt length s, caching the system prompt introduces an additional computational overhead of $4Lhsd$ FLOPs per newly generated token—arising from both Q-K dot products and V-weighted summations.

In contrast, SysVec requires only a minimal computational overhead involving a single matrix addition operation of dimension $[1, d]$, which amounts to roughly d additional FLOPs per token. Thus, SysVec not only reduces context length but also introduces significantly less per-token computational overhead compared to relying solely on cached system prompts.

Empirical Results. We empirically validated this advantage by measuring inference times on the Llama-3-8B model across various applications, comparing the performance of textual system prompts and SysVec. The detailed results are summarized in Table 5.

In scenarios where only one new token is generated (a scenario approximating the model's prefill speed, representing the worst-case efficiency), inference times using textual system prompts ranged from 0.053s (shortest prompt, 261 tokens in the Paimon task) to 0.106s (longest prompt, 996 tokens in the Stoic task) per query. In contrast, SysVec consistently reduced inference times to approximately 0.015s per query, reflecting a significant and stable reduction irrespective of the system prompt length.

Furthermore, in a more typical scenario with a maximum token generation length of 4096, SysVec similarly outperformed textual prompts, reducing the average inference time per query across all tasks. For example, on the Stoic task with the longest textual system prompt, SysVec reduced the inference time from 5.166s to 2.784s, achieving a substantial improvement in computational efficiency.

To quantify the cost-effectiveness of SysVec more explicitly, we estimated an economic break-even point—the number of inference queries required for SysVec's upfront optimization cost to be offset by its reduced inference computation. In the worst-case scenario (max_new_tokens=1), SysVec achieves computational cost benefits after approximately 18,000–44,000 queries. However, in the more realistic scenario of generating longer content (max_new_tokens=4096), SysVec reaches this break-even point much earlier, typically after only around 700–3200 queries. Given the practical application scenarios, especially where protecting proprietary system prompts and contexts is crucial, reaching this query volume is easily achievable. Even if considering solely the security benefits, the additional one-time computational cost (less than half an hour) remains fully acceptable.

4.6 Adaptive Attack

We also consider the scenario where the attacker knows the defense details and designs adaptive attacks. Specifically, in this scenario, we assume that the attacker knows the target model uses SysVec to protect the system prompt and understands the technical details of SysVec. We also assume that the adversary knows the specific model and architecture of the target LLM and has the capability to deploy the same model locally; the adversary can also send unlimited queries to the target model and access the output logits distribution for each query.

Under this strong threat scenario, an attacker who spares no cost might adopt an intuitive adaptive attack strategy: initialize a shadow vector $\hat{\mathbf{v}}$ on the attacker's local shadow model. We assume the attacker also knows the deployment layer position of the shadow vector $\hat{\mathbf{v}}$, i.e., ℓ. Then send multiple requests to the target model and record the returned logits, while computing the logits of the local model under the same inputs, then train the shadow vector $\hat{\mathbf{v}}$ by computing the KL divergence between the two sets of values. Ultimately, the attacker expects the shadow vector $\hat{\mathbf{v}}$ to approximate the SysVec $\mathbf{v}$.

Furthermore, considering that the attacker may wish to obtain the original textual system prompt corresponding to SysVec, we refer to the method in Morris et al. [23] to convert the shadow vector $\hat{\mathbf{v}}$ back to text form, obtaining an approximate text prompt $\hat{\mathbf{s}}$. Finally, we compute the RUS-score when using the shadow vector $\hat{\mathbf{v}}$ and when using the approximate text prompt $\hat{\mathbf{s}}$ respectively, to evaluate the utility of the stolen system prompt. We implemented

Table 5: Time Costs.

Application	D&D	Paimon	ML	Advisor	Stoic
Textual system prompt Avg. inference time (s/query), max_new_tokens=1	0.083	0.053	0.082	0.054	0.106
SysVec Avg. inference time (s/query), max_new_tokens=1	0.015	0.016	0.015	0.015	0.015
Textual system Prompt Avg. inference time (s/query), max_new_tokens=4096	1.730	1.946	2.890	4.573	5.166
SysVec Avg. inference time (s/query), max_new_tokens=4096	1.176	1.461	2.233	2.566	2.784
SysVec training time (s)	1540.2	1555.2	1655.2	1700.1	1705.1
Cost-benefit balance threshold, max_new_tokens=1, (worst case)	22650	40926	24705	43593	18738
Cost-benefit balance threshold,max_new_tokens=4096 (normal case)	2781	3207	2520	848	716

the adaptive attack in Llama2-7B-chat model and the "Paimon" system prompt, using 5,000 requests to compute the shadow vector $\hat{\mathbf{v}}$, with the generation method consistent with the previous sections.

We first evaluate the RUS-score of the model directly using $\hat{\mathbf{v}}$. The RUS-score is 6.89, while the original textual prompt and the SysVec vector $\mathbf{v}$ achieve scores of 7.76 and 7.80, respectively. Next, following the method and pretrained weights provided in Morris et al. [23], we invert $\hat{\mathbf{v}}$ into textual prompts $\hat{s}$. We generate 20 different $\hat{\mathbf{s}}$ and compute their average RUS Score, which is 3.35.

We find that such attacks can hardly obtain practically effective system prompts. After examining the results, we find this is mainly because existing model inversion methods have strong limitations on the length of inverted content, with their maximum generation length being much smaller than the system prompts we use. If attackers want to obtain usable textual system prompts, they need to implement stronger model inversion methods. While directly using the shadow vector $\hat{\mathbf{v}}$ has a relatively higher RUS-score, it is still lower than scenarios using textual system prompts and SysVec. In fact, similar "model distillation attacks" can theoretically bypass any defense method—as long as the target model follows the system prompt well. Considering that the current adaptive attack is already built on a very strong threat model, requiring attackers to have gray-box access, know the implementation details of the defense, and have sufficient computational resources, we can consider that SysVec at least significantly increases the attack difficulty and cost, while the requirement for multiple model queries also increases the attacker's exposure risk.

4.7 Evaluating SysVec under Prefill Attack

Although typical prompt leaking attack scenarios assume a black-box setting—where attackers have limited access to model internals—we further explore an extra threat model in which the attacker possesses stronger capabilities

, such as obtaining and inspecting model parameters directly. We consider this stronger scenario because practical deployments of LLM services must anticipate and safeguard against security threats beyond standard contexts. For instance, attackers might employ side-channel attacks or other sophisticated techniques to extract detailed information about the model parameters and internal architectures, significantly expanding their potential attack surface. Thus, it is crucial to evaluate whether SysVec remains robust even in scenarios where attackers have elevated access privileges.

A representative method in white-box scenarios is GCG [49], originally proposed for jailbreak attacks. GCG involves backpropagating through the victim model to iteratively optimize an adversarial suffix, compelling the LLM to start responses with specific affirmative prefixes such as "Sure, here is …," thereby leaking sensitive information. However, GCG's applicability to prompt leaking attacks is somewhat unrealistic. If attackers already possess white-box access enabling direct optimization through backpropagation, they would typically have no need to resort to indirect leakage attacks, as they could directly inspect or extract the system prompts and model internals from the model parameters.

Therefore, we adopt an attack strategy that utilizes the core idea of GCG—forcing specific response prefixes—without requiring full white-box access: Prefill Attack [46]. Prefill Attack directly manipulates the logits of LLM outputs, explicitly forcing the model to begin responses with a predefined target string, such as "Sure, here is my current system prompt:". Unlike GCG, Prefill Attack does not require iterative optimization via backpropagation at attack time, making it a practical attack strategy that can be simply combined with existing prompt leaking attacks. Furthermore, Prefill Attack achieves a similar goal as GCG, guaranteeing that the victim model starts responses in an attacker-desired manner, but in a more practically deployable form.

We empirically evaluate Prefill Attack on Llama2-7B and Mistral-7B models, including scenarios where Prefill Attack is combined with existing adversarial suffix strategies. Our results indicate that Prefill Attack slightly increasing leakage scores relative to non-Prefill conditions when combine with parts of attacks. Despite this enhanced attack effectiveness against traditional defenses, SysVec robustly mitigates the attack, maintaining significantly lower leakage scores across all evaluated attack combinations.

Additionally, for research purposes, although the attack scenario is not realistic, we also considered the scenario of directly using GCG attacks. We set the optimization target of GCG to "Sure, Here is my system prompt:". The results are shown in Table.7. We can observe that GCG also cannot effectively attack our method. This again demonstrates the effectiveness of our core idea: removing the system prompt from the context can significantly reduce the risk of leakage.

Table 6: The PLS score (↓) between the leaked system prompt and actual system prompt of three LLMs when using different attack combine with Prefill Attack and defense strategies. A higher score indicates that the leaked content is closer to the actual system prompt. The best results are highlighted in bold. Prefill Attack Results.

Model	Llama2-7B-chat-hf				
Defense	No Defense	Reminder [44]	In-Context [40]	Isolation [41]	SysVec (Ours)
Naive Attack [21]	$4.77_{\pm 1.66}$	$4.59_{\pm 1.62}$	$4.78_{\pm 1.69}$	$4.30_{\pm 1.52}$	$\mathbf{1.69}_{\pm 0.97}$
Ignore Attack [5]	$4.52_{\pm 1.54}$	$4.49_{\pm 1.57}$	$4.49_{\pm 1.51}$	$4.30_{\pm 1.53}$	$\mathbf{1.87}_{\pm 0.93}$
Completion Attack [41]	$4.85_{\pm 1.65}$	$4.93_{\pm 1.64}$	$5.53_{\pm 1.59}$	$4.83_{\pm 1.70}$	$\mathbf{1.44}_{\pm 0.65}$
Ignore-Completion Attack	$4.64_{\pm 1.52}$	$4.44_{\pm 1.61}$	$5.30_{\pm 1.23}$	$4.77_{\pm 1.64}$	$\mathbf{1.55}_{\pm 0.84}$
Remember-the-Start Attack	$5.00_{\pm 1.63}$	$4.96_{\pm 1.59}$	$5.34_{\pm 1.52}$	$4.43_{\pm 1.76}$	$\mathbf{1.61}_{\pm 1.13}$
Ignore-Remember Attack	$5.32_{\pm 1.61}$	$4.95_{\pm 1.64}$	$5.41_{\pm 1.90}$	$5.55_{\pm 1.46}$	$\mathbf{2.30}_{\pm 0.91}$
Completion-Remember Attack	$5.23_{\pm 1.71}$	$5.50_{\pm 1.65}$	$5.41_{\pm 1.77}$	$5.47_{\pm 1.52}$	$\mathbf{1.98}_{\pm 0.66}$
Ignore-Completion-Remember Attack	$5.11_{\pm 1.55}$	$4.36_{\pm 1.70}$	$4.65_{\pm 1.91}$	$5.24_{\pm 1.45}$	$\mathbf{2.04}_{\pm 0.69}$
PLeak [12]	$6.42_{\pm 1.57}$	$6.25_{\pm 1.42}$	$6.07_{\pm 1.74}$	$6.37_{\pm 1.61}$	$\mathbf{1.50}_{\pm 0.76}$
PLeak-single	$4.88_{\pm 1.50}$	$4.92_{\pm 1.54}$	$5.02_{\pm 1.52}$	$4.37_{\pm 1.57}$	$\mathbf{1.32}_{\pm 0.51}$
Model	Mistral-7B-Instruct				
Defense	No Defense	Reminder [44]	In-Context [40]	Isolation [41]	SysVec (Ours)
Naive Attack [21]	$5.76_{\pm 1.60}$	$6.04_{\pm 1.54}$	$6.07_{\pm 1.57}$	$5.47_{\pm 1.81}$	$\mathbf{1.83}_{\pm 0.76}$
Ignore Attack [5]	$5.98_{\pm 1.66}$	$5.81_{\pm 1.63}$	$5.62_{\pm 1.70}$	$5.35_{\pm 1.66}$	$\mathbf{2.09}_{\pm 1.02}$
Completion Attack [41]	$6.46_{\pm 1.54}$	$6.26_{\pm 1.60}$	$6.89_{\pm 1.08}$	$6.0_{\pm 1.38}$	$\mathbf{1.88}_{\pm 0.77}$
Ignore-Completion Attack	$6.00_{\pm 1.55}$	$6.01_{\pm 1.61}$	$6.27_{\pm 1.60}$	$5.77_{\pm 1.76}$	$\mathbf{1.90}_{\pm 0.69}$
Remember-the-Start Attack	$6.63_{\pm 1.32}$	$6.61_{\pm 1.29}$	$6.63_{\pm 1.25}$	$6.73_{\pm 1.51}$	$\mathbf{1.62}_{\pm 0.71}$
Ignore-Remember Attack	$6.89_{\pm 1.34}$	$6.74_{\pm 1.31}$	$6.28_{\pm 1.49}$	$7.08_{\pm 1.31}$	$\mathbf{1.56}_{\pm 0.61}$
Completion-Remember Attack	$6.52_{\pm 1.39}$	$6.17_{\pm 1.42}$	$6.37_{\pm 1.41}$	$6.59_{\pm 1.43}$	$\mathbf{1.84}_{\pm 0.72}$
Ignore-Completion-Remember Attack	$6.97_{\pm 1.29}$	$6.94_{\pm 1.27}$	$6.73_{\pm 1.43}$	$7.01_{\pm 1.26}$	$\mathbf{1.84}_{\pm 0.60}$
PLeak [12]	$7.51_{\pm 1.31}$	$7.15_{\pm 1.73}$	$7.52_{\pm 1.65}$	$7.91_{\pm 1.83}$	$\mathbf{1.78}_{\pm 0.52}$
PLeak-single	$5.62_{\pm 1.87}$	$5.47_{\pm 1.74}$	$5.66_{\pm 1.65}$	$4.77_{\pm 1.85}$	$\mathbf{1.69}_{\pm 0.65}$

Table 7: Effectiveness of SysVec against GCG Attack

Model	NoDef	Reminder	InContext	Isolation	SysVec
Llama2	$6.52_{\pm 1.98}$	$5.95_{\pm 2.31}$	$6.71_{\pm 2.45}$	$6.18_{\pm 3.09}$	$\mathbf{1.57}_{\pm 1.12}$
Mistral	$7.29_{\pm 2.43}$	$7.51_{\pm 1.62}$	$6.95_{\pm 2.28}$	$7.49_{\pm 1.53}$	$\mathbf{1.55}_{\pm 1.71}$
Llama3	$8.11_{\pm 1.85}$	$6.89_{\pm 2.27}$	$7.88_{\pm 1.81}$	$6.45_{\pm 1.99}$	$\mathbf{2.24}_{\pm 0.87}$

4.8 SysVec Mitigates Forgetting in Multi-turn Conversations

Compared to textual system prompts, SysVec has another advantage: it might do a better job enforcing the system prompt in long-context scenarios. This forgetting problem often arises when users interact with a ChatBot over multiple rounds. As the conversation grows longer, the model may produce answers that fail to meet requirements or drop in quality. To simulate this scenario, we first generated ten new questions that lay outside the training and test sets. We then appended varying numbers of these new questions and their responses to the conversation history. After that, we computed the Response Utility Score on the test set for each scenario. For example, to measure the model's quality in the third round, we provided either the textual system prompt or SysVec plus two rounds of dialogue history, followed by the samples from the test set.

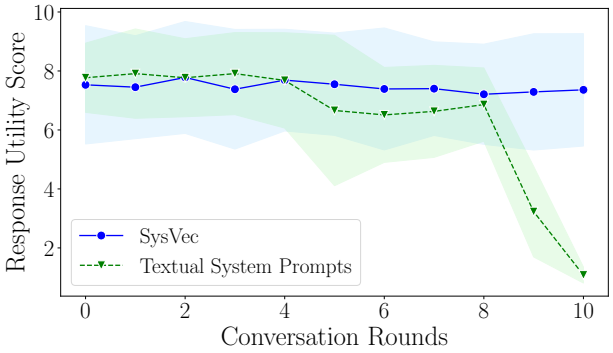

Figure 5: The trend of the Response Utility Score changes with the increasing conversation rounds in multi-turn conversations.

Figure 5 shows the average and standard deviation of the Response Utility Score at different rounds for the Llama2-7B-chat-hf model. We observe a downward trend in response quality when using a textual system prompt as the number of conversation rounds increases. This decline is especially severe for longer contexts (when

Rounds > 8). Once *Rounds* > 10, the Response Utility Score approaches 1 (the lowest score). At *Rounds* = 15 (not plotted), the textual system prompt's Response Utility Score is 1.0 ± 0, indicating every sample received the minimum rating. In that situation, the model randomly repeats irrelevant content. By contrast, SysVec achieves a Response Utility Score of 7.74 ± 1.51 at *Rounds* = 15, and the response quality remains stable regardless of the conversation length. These results demonstrate that SysVec is more robust in maintaining consistent model behavior across extended conversations, effectively addressing the common challenge of system prompt forgetting in multi-rounds conversation scenarios.

4.9 Can SysVec be flexibly adjusted?

An important advantage of textual system prompts is that they can be intuitively and quickly adjusted. However, for SysVec, each modification requires retraining, which increases limitations on its application. To alleviate this problem, we considered the following two improvement methods:

Post-training of SysVec. We notice that in most cases, adjustments to system prompts are only minor modifications or additions, where most of the content may remain unchanged. In this case, can the SysVec corresponding to the system prompt before adjustment be considered as a good initialization point? Based on this idea, we propose a corresponding adjustment scheme: when the text system prompt $\mathbf{s}$ changes are limited, first, initialize the SysVec $\mathbf{v}$ with the current SysVec $\mathbf{v}_{old}$. Treat the output $\mathbf{y}$ produced by the original system prompt as $\mathbf{y}_{\text{lose}}$ and the output produced after the prompt is modified as $\mathbf{y}_{\text{win}}$, then optimize SysVec on this pairwise signal.

We conducted experiments on the Llama3-8B-Instruct model and the "Stoic" System Prompt. We first randomly removed sentences accounting for about 20% of the total length of the system prompt, then trained a SysVec with the remaining 80% content. Then, we used the SysVec obtained in this step as initialization, with the complete "Stoic" System Prompt as the target, to train a new SysVec, and recorded the time required to reach loss convergence and achieve performance comparable to the textual system prompt. We found that the average time required for 10 attempts was about 913.8s, a reduction of about 46% compared to training time with random initialization.

Joint use with textual system prompt. Another straightforward idea is to add additional textual system prompts while using SysVec. Although this may lead to the newly added textual system prompt being leaked, it can also serve as an emergency adjustment strategy. We used the same model and system prompt as in the previous experiment, and added an extra system prompt that asks for French answers. Compliance rates are 92.5% (textual prompt) and 91.0% (SysVec), showing the limited impact of the additional prompt.

While both strategies can alleviate the problem of SysVec's inflexibility in adjustment, they both come with certain limitations, revealing an important limitation of SysVec.

4.10 Ablation Studies

We conducted an ablation study on the Llama-3 model to investigate the impact of two key hyperparameters: the injection layer l where SysVec is added, and the multiplier α used when applying

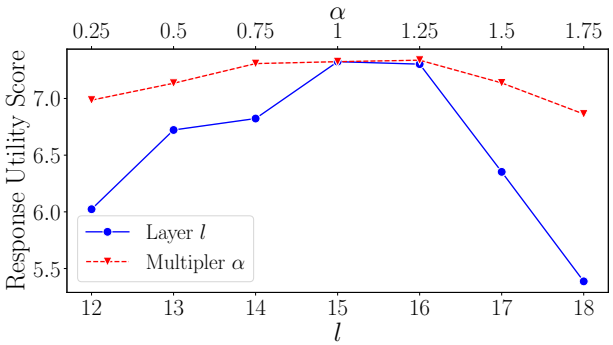

Figure 6: The Response Utility Score of different hyperparameters.

SysVec. The results are shown in Figure 6. We observe that SysVec's performance remains largely stable when using hyperparameters close to the default settings. However, unlike representation vectors extracted via contrastive learning in prior RepE methods [26], our extracted SysVec does not exhibit a clearly linear influence on the model behavior as α increases. On the other hand, injecting SysVec at an inappropriate layer can significantly affect performance.

Nevertheless, these observations may not imply a high tuning cost. We find that the optimal hyperparameter configuration often generalizes across tasks—i.e., the best layer and multiplier for one system prompt tend to remain effective for others. This aligns with findings from prior work on Representation Engineering [3, 26, 48], which suggests that the optimal injection layer l is more dependent on the model architecture than on the specific task. Moreover, these studies report that even across different models, the optimal l tends to fall within a consistent range. For example, in LLMs with 48 layers, the most effective injection layers are typically between the 10th and 18th layers.

4.11 Discussion on Potential Additional Defenses

Table 8: Extra Defenses

Model	Llama-3-8B-Instruct			
Defense	No Defense	PPL	Paraphrasing	SysVec (Ours)
Naive Attack [21]	$4.16_{\pm 1.73}$	$4.07_{\pm 1.86}$	$4.13_{\pm 1.68}$	$\mathbf{2.83}_{\pm 1.36}$
Ignore Attack [5]	$4.83_{\pm 1.82}$	$4.28_{\pm 2.61}$	$4.91_{\pm 1.64}$	$\mathbf{3.01}_{\pm 1.48}$
Completion Attack [41]	$4.12_{\pm 1.64}$	$4.32_{\pm 1.97}$	$4.30_{\pm 1.50}$	$\mathbf{2.88}_{\pm 1.47}$
I-C Attack	$4.29_{\pm 1.54}$	$4.37_{\pm 1.61}$	$4.16_{\pm 1.16}$	$\mathbf{3.05}_{\pm 1.51}$
Remember Attack	$7.21_{\pm 1.68}$	$7.21_{\pm 1.19}$	$7.01_{\pm 1.86}$	$\mathbf{3.21}_{\pm 1.61}$
I-R Attack	$8.43_{\pm 1.12}$	$7.98_{\pm 1.60}$	$7.95_{\pm 1.26}$	$\mathbf{3.56}_{\pm 1.69}$
C-R Attack	$5.42_{\pm 1.82}$	$5.49_{\pm 1.74}$	$5.61_{\pm 1.82}$	$\mathbf{3.30}_{\pm 1.45}$
I-C-R Attack	$7.23_{\pm 1.21}$	$7.05_{\pm 1.50}$	$7.02_{\pm 1.85}$	$\mathbf{3.45}_{\pm 1.62}$
PLeak [12]	$7.53_{\pm 1.84}$	$4.67_{\pm 2.55}$	$7.14_{\pm 1.67}$	$\mathbf{2.02}_{\pm 1.86}$

Prior work has also explored alternative defenses designed to mitigate adversarial attacks against LLMs [12], such as those originally proposed to address jailbreak attacks [2, 13]. Among these approaches, perplexity-based defenses (PPL) and paraphrasing-based

defenses are widely employed [2, 13]. The perplexity-based defense attempts to filter out adversarial inputs by calculating perplexity scores and rejecting inputs with abnormally high perplexity. The paraphrasing-based defense uses an LLM to paraphrase inputs, aiming to neutralize any potentially harmful or adversarial elements. However, these methods are primarily designed to address adversarial attacks. Specifically, perplexity-based methods exploit the unusually high perplexity typical of adversarial suffixes, whereas paraphrasing-based methods depend on adversarial inputs lacking meaningful semantic coherence.

Prompt leaking attacks often do not exhibit these specific adversarial characteristics, making these existing defense strategies unsuitable. We conducted experiments to empirically verify this observation, reporting the PLS scores for these additional defense methods across various attack scenarios in Table 8.

As shown, both perplexity-based and paraphrasing-based defenses exhibit limited effectiveness against prompt leaking attacks. Although these methods sometimes show partial effectiveness—for example, the perplexity-based defense achieves a slightly lower leakage score under PLeak due to its adversarial characteristics—their overall performance remains unsatisfactory. These results indicate that despite occasional improvements, both defenses struggle to counter prompt leaking attacks effectively. In contrast, SysVec consistently achieves significantly lower PLS scores across all attack settings, including the strong and optimized PLeak attack, demonstrating its superior robustness and generalization.

5 Ethical Impact Statement

In this work, we introduced both a novel attack method that exposes vulnerabilities in current LLM applications and a defensive solution that fundamentally addresses these security concerns. The proposed SysVec solution could have broad positive implications for the LLM ecosystem by enabling more secure deployment of AI systems. And we acknowledge that making the Remember-the-start attack public could temporarily increase the risk of system prompt leakage until protective measures are widely adopted, but we believe the benefits of disclosure outweigh the risks, as it promotes awareness and encourages the development of more robust security measures.

We have followed responsible disclosure procedures by notifying affected companies about the Remember-the-Start attack vulnerability before public disclosure, allowing them to implement necessary safeguards.

6 Limitations

SysVec has several limitations. Compared to textual system prompts, SysVec inevitably introduces additional deployment costs and reduces flexibility during adjustments. Although we propose corresponding methods to mitigate this issue, in terms of convenience and flexibility, the textual system prompt still wins. Another limitation is that our method requires defenders to have white-box access to the model. In most scenarios, this assumption is realistic since the defense is applied by the proprietary owner of the LLM (e.g., OpenAI), who by default has access to the model. However, defenders may lack such white-box access in certain scenarios, such as developing LLM-based applications using third-party model APIs. In these situations, SysVec can be extended to a black-box setting by exposing an SFT-like API. Specifically, users can provide textual system prompts, after which the model owner trains and deploys the corresponding SysVec. Lastly, similar to most defense mechanisms, SysVec cannot provide absolute security guarantees. Attackers may circumvent SysVec by developing advanced attack methods. For instance, our threat model assumes attackers only have black-box access to the model. However, recent studies, such as LLMmap [27], have demonstrated methods for identifying specific LLMs, potentially enabling attackers to perform white-box attacks on applications that use open-source LLM backbones.

7 Acknowledgements

The Authors acknowledge the National Artificial Intelligence Research Resource (NAIRR) Pilot for contributing to this research result. Changjiang Li and Ting Wang are partially supported by the National Science Foundation under Grant No. 2405136 and 2406572. The views and conclusions contained in this paper are those of the authors and should not be interpreted as representing any funding agencies.

8 Conclusion

In this paper, we introduce SysVec, a novel approach that fundamentally addresses the prompt leakage vulnerability in LLMs by encoding system prompts as internal representation vectors rather than raw text. Our work makes two key contributions: First, we demonstrate the ongoing vulnerability of current LLMs through our proposed Remember-the-Start attack, which successfully extracts system prompts from various LLM-based applications by helping LLMs restore their context repetition capability. Second, we present SysVec as a principled defense that moves system prompts outside the context entirely, preventing leakage while preserving or even enhancing model utility. Our findings highlight the importance of rethinking how system prompts are implemented in LLM applications and demonstrate that encoding them as internal representations offers a promising direction for both security and performance.

References

[1] Anthropic. 2024. Claude 3.5 Sonnet. (2024). https://www.anthropic.com/news/claude-3-5-sonnet
[2] Bochuan Cao, Yuanpu Cao, Lu Lin, and Jinghui Chen. 2023. Defending against alignment-breaking attacks via robustly aligned llm. *arXiv preprint arXiv:2309.14348* (2023).
[3] Yuanpu Cao, Tianrong Zhang, Bochuan Cao, Ziyi Yin, Lu Lin, Fenglong Ma, and Jinghui Chen. 2024. Personalized Steering of Large Language Models: Versatile Steering Vectors Through Bi-directional Preference Optimization. *arXiv preprint arXiv:2406.00045* (2024).
[4] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2024. StruQ: Defending against prompt injection with structured queries. *arXiv preprint arXiv:2402.06363* (2024).
[5] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, and Chuan Guo. 2024. Aligning llms to be robust against prompt injection. *arXiv preprint arXiv:2410.05451* (2024).
[6] Eunbi Choi, Yongrae Jo, Joel Jang, and Minjoon Seo. 2022. Prompt injection: Parameterization of fixed inputs. *arXiv preprint arXiv:2206.11349* (2022).
[7] Haodong Duan, Jueqi Wei, Chonghua Wang, Hongwei Liu, Yixiao Fang, Songyang Zhang, Dahua Lin, and Kai Chen. 2023. Botchat: Evaluating llms' capabilities of having multi-turn dialogues. *arXiv preprint arXiv:2310.13650* (2023).
[8] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[9] Dongyoung Go, Tomasz Korbak, Germán Kruszewski, Jos Rozen, Nahyeon Ryu, and Marc Dymetman. 2023. Aligning language models with preferences through f-divergence minimization. *arXiv preprint arXiv:2302.08215* (2023).

[10] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).

[11] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[12] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. Pleak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3600–3614.

[13] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline Defenses for Adversarial Attacks Against Aligned Language Models. arXiv:2309.00614 [cs.LG]

[14] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[15] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. Llms get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120* (2025).

[16] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. 2024. Inference-time intervention: Eliciting truthful answers from a language model. *Advances in Neural Information Processing Systems* 36 (2024).

[17] Sheng Liu, Haotian Ye, Lei Xing, and James Zou. 2023. In-context vectors: Making in context learning more effective and controllable through latent space steering. *arXiv preprint arXiv:2311.06668* (2023).

[18] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, et al. 2023. Prompt Injection attack against LLM-integrated Applications. *arXiv preprint arXiv:2306.05499* (2023).

[19] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2023. Prompt injection attacks and defenses in llm-integrated applications. *arXiv preprint arXiv:2310.12815* (2023).

[20] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1831–1847.

[21] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1831–1847.

[22] Ilya Loshchilov, Frank Hutter, et al. 2017. Fixing weight decay regularization in adam. *arXiv preprint arXiv:1711.05101* 5 (2017).

[23] John X Morris, Wenting Zhao, Justin T Chiu, Vitaly Shmatikov, and Alexander M Rush. 2023. Language model inversion. *arXiv preprint arXiv:2311.13647* (2023).

[24] Ha-Thanh Nguyen. 2023. A Brief Report on LawGPT 1.0: A Virtual Legal Assistant Based on GPT-3. *arXiv preprint arXiv:2302.05729* (2023).

[25] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]

[26] Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. 2023. Steering llama 2 via contrastive activation addition. *arXiv preprint arXiv:2312.06681* (2023).

[27] Dario Pasquini, Evgenios M Kornaropoulos, and Giuseppe Ateniese. 2024. Llmmap: Fingerprinting for large language models. *arXiv preprint arXiv:2407.15847* (2024).

[28] Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527* (2022).

[29] Julien Piet, Maha Alrashed, Chawin Sitawarin, Sizhe Chen, Zeming Wei, Elizabeth Sun, Basel Alomair, and David Wagner. 2024. Jatmo: Prompt injection defense by task-specific finetuning. In *European Symposium on Research in Computer Security*. Springer, 105–124.

[30] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. 2023. Fine-tuning aligned language models compromises safety, even when users do not intend to! *arXiv preprint arXiv:2310.03693* (2023).

[31] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2024).

[32] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[33] Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. 2024. Optimization-based prompt injection attack to llm-as-a-judge. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 660–674.

[34] Nishant Subramani, Nivedita Suresh, and Matthew E Peters. 2022. Extracting latent steering vectors from pretrained language models. *arXiv preprint arXiv:2205.05124* (2022).

[35] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[36] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large language models in medicine. *Nature medicine* (2023), 1–11.

[37] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[38] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J Vazquez, Ulisse Mini, and Monte MacDiarmid. 2023. Activation addition: Steering language models without optimization. *arXiv e-prints* (2023), arXiv–2308.

[39] Haoran Wang and Kai Shu. 2023. Backdoor activation attack: Attack large language models using activation steering for safety-alignment. *arXiv preprint arXiv:2311.09433* (2023).

[40] Zeming Wei, Yifei Wang, and Yisen Wang. 2023. Jailbreak and guard aligned language models with only few in-context demonstrations. *arXiv preprint arXiv:2310.06387* (2023).

[41] Simon Willison. 2024. Delimiters won't save you from prompt injection.

[42] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. 2023. BloombergGPT: A Large Language Model for Finance. arXiv:2303.17564 [cs.LG]

[43] Haoran Yang, Yumeng Zhang, Jiaqi Xu, Hongyuan Lu, Pheng Ann Heng, and Wai Lam. 2024. Unveiling the generalization power of fine-tuned large language models. *arXiv preprint arXiv:2403.09162* (2024).

[44] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2023. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197* (2023).

[45] Jiahao Yu, Yuhang Wu, Dong Shu, Mingyu Jin, and Xinyu Xing. 2023. Assessing prompt injection risks in 200+ custom gpts. *arXiv preprint arXiv:2311.11538* (2023).

[46] Hangfan Zhang, Zhimeng Guo, Huaisheng Zhu, Bochuan Cao, Lu Lin, Jinyuan Jia, Jinghui Chen, and Dinghao Wu. 2023. On the Safety of Open-Sourced Large Language Models: Does Alignment Really Prevent Them From Being Misused? *arXiv preprint arXiv:2310.01581* (2023).

[47] Zhexin Zhang, Junxiao Yang, Pei Ke, Fei Mi, Hongning Wang, and Minlie Huang. 2023. Defending large language models against jailbreaking attacks through goal prioritization. *arXiv preprint arXiv:2311.09096* (2023).

[48] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. 2023. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405* (2023).

[49] Andy Zou, Zifan Wang, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).